

CONTENTS

[Part 1, Architecture Design](#)

[Part 2, Working Code](#)

[Part 3, Design Decisions](#)

Architecture View

This document is the long form report for the take home assessment. It is organised in three parts that match the deliverables section of the assessment PDF.

1. Architecture Design. The agent architecture, the planner, the scheduler, MCP orchestration, state, long running task handling, bulk execution, failure recovery, retries, observability.
2. Working Code. What is actually runnable, how planning works, how agent routing works, how MCP tools are called, how multi step workflows are stitched together. A sample UI walkthrough with screenshots from a live run.
3. Design Decisions. Why this architecture and not another, the tradeoffs that mattered, what alternative approaches were considered and avoided, what scaling looks like at one million documents, and what would still need to change for a real production rollout.

The codebase lives under `hermes_framework/`. The MCP tools and the 36,000 document corpus live under `Assessment/`. The README has the quick start and the API surface. This document covers the architecture end to end and explains the design thinking behind every choice.

Part 1, Architecture Design

Agent architecture

The system is a small set of specialised agents that all sit behind one FastAPI process. The top level orchestrator is the only stateful component. Every other agent is a function call that returns when it is done. There is no shared mutable state across agents apart from the SQLite store, which is the durability boundary.

There are five agents in total.

The Interrogator runs first. It decides whether the user request is specific enough to plan against or whether it needs one or two clarifying questions. It returns either a `proceed` signal or a list of questions. The questions get persisted as rows and emitted as SSE events so the UI can render them. The session pauses on `AWAITING_ANSWER` until the user answers.

The Planner is the brain. It takes the user request plus the available container list plus the live MCP tool schemas, and emits a DAG of typed tasks via a single `emit_plan` tool call. The DAG has explicit dependencies and an explicit task kind on every node. The Planner runs with Claude Sonnet 4.6 in adaptive thinking mode, and its thinking deltas are streamed to the UI live so the user sees the reasoning as it happens. Total Planner output is a structured plan, not text.

The Scheduler is the executor. It walks the DAG in topological waves up to eight concurrent tasks per wave, dispatches each task to the right worker, handles retries with exponential backoff on transient failure, and marks downstream tasks SKIPPED when an upstream task fails. After a failure the orchestrator can request one bounded replan with the failure context fed back to the Planner.

The Workers do the actual work. There are three of them and they are deliberately small. The ToolWorker makes one direct MCP call. The CodeWorker generates a small Python script via Claude, validates it, and runs it in a sandbox subprocess that calls MCP tools and streams progress back. The SubAgentWorker runs a child Claude session for artifacts like HTML dashboards or for the final answer composition.

The Synthesizer is technically a SubAgentWorker invocation but it has its own system prompt that is strict about markdown style. It composes the final user facing answer from upstream task outputs, including clickable absolute URLs to any artifacts.

Apart from the SQLite store and a per session asyncio event bus, the agents do not share state. Each is one function call that returns its result. The shape is intentional, since each agent can then be invoked and tested in isolation without setting up the rest of the system.

Planning engine

The Planner is a single Claude call. The prompt is large and stable, the output is a structured tool call.

```
PLANNER_SYSTEM (cache enabled, ephemeral)
+ EMIT_PLAN_TOOL schema
+ user message
+ container_id and available_containers list
+ LIVE MCP TOOL SCHEMAS section (rebuilt every call)
+ optional prior_failure context (only on replan)
```

The model emits a plan in this shape.

```

{
  "goal": "one sentence restatement",
  "tasks": [
    {
      "id": "T1",
      "kind": "CODE_TRANSFORM",
      "title": "Find and translate all financial documents",
      "depends_on": [],
      "spec": {
        "code_intent": "Fetch metadata from every container in parallel, ...",
        "expected_output_schema": {"successful": "int", "failed": "int"}
      }
    },
    {
      "id": "T2",
      "kind": "SYNTHESIZE",
      "title": "Report",
      "depends_on": ["T1"],
      "spec": {"instructions": "...", "inputs_from": ["T1"]}
    }
  ]
}

```

The Planner relies on three specific configuration choices worth calling out.

First, `thinking={"type": "adaptive"}` instead of a fixed token budget. Adaptive thinking lets the model choose how much to think per query. A simple question gets a short plan with no thinking. A complex "translate all financial documents across all containers and report breakdowns" gets visible reasoning that streams to the UI as it happens.

Second, `tool_choice={"type": "auto"}` not forced tool use. Adaptive thinking is incompatible with forced tool calls. With a single tool surface (`emit_plan`) and a strict system prompt, the model picks the tool in practice every time. If it ever does not, the orchestrator falls back to a clear error rather than silently misbehaving.

Third, `cache_control={"type": "ephemeral"}` on the system prompt and tool schema. The system prompt is several kilobytes of strategy guidance and decision tables, and it is byte stable across calls in the same process. Prompt caching makes the second and subsequent Planner calls in a session pay roughly ten percent of the prefix cost.

The dynamic part of the prompt is the LIVE MCP TOOL SCHEMAS section. It is built at runtime from the actual `list_tools()` response of the connected MCP server. If a tool's input schema changes on the server, the next plan reflects it without code changes here.

Task decomposition logic

The Planner decomposes user requests into typed tasks. Six task kinds cover the entire decision space.

TOOL_CALL	one direct MCP call with known args (up to 20 doc IDs)
RAG_QUERY	one aiagent Q and A call
CODE_TRANSFORM	Python script generated and executed in the sandbox
BULK_TOOL_CALL	explicit bulk MCP call, also via generated code
SUBAGENT	child Claude session for an artifact such as HTML
SYNTHESIZE	final user facing answer composition

The rule of thumb. If the task iterates over docs or spans containers, it is `CODE_TRANSFORM`. If it is a single MCP call with known args, it is `TOOL_CALL`. If it composes a response from upstream data, it is `SUBAGENT` or `SYNTHESIZE`.

There is one hard structural invariant that drives the whole scalability story. A `TOOL_CALL` cannot carry more than twenty document IDs. Beyond that, iteration must live in `CODE_TRANSFORM`. The Planner prompt forbids this with worked examples and an explicit note. The `validate_and_repair` function in `app/engine/planner.py` is the load bearing defence. It scans the emitted plan, counts document IDs in every `TOOL_CALL`, and rewrites violators as `CODE_TRANSFORM` with a descriptive `code_intent`. It emits a `plan.repaired` SSE event so the user sees the rewrite happened. The same function also rejects task IDs that reference unknown tool names, detects dependency cycles, and catches duplicate task IDs.

A typical translation plan looks like this.

```
T1 CODE_TRANSFORM "Find and translate all financial documents"
  depends_on: []
  code_intent:
    - fetch metadata from every container in parallel
    - filter docs where category == 'financial'
    - group ids by container
    - call translate per container in parallel, one bulk call each
    - emit per chunk progress, return aggregated counts

T2 SYNTHESIZE "Report results"
  depends_on: [T1]
  inputs_from: [T1]
```

Two tasks. Three LLM calls total (Interrogator, Planner, Synthesizer), plus one Code generation LLM call inside T1. The actual translation work is one bulk MCP call per container, which the MCP server handles internally with `Semaphore(200)`. The Planner never sees the document IDs.

Execution engine

The Scheduler is the heart of execution. It is a topological wave executor with a concurrency cap.

```
while any task is PENDING:
    ready = [t for t in plan.tasks
              if t.status == PENDING and all deps SUCCEEDED]
    if not ready:
        mark remaining PENDING as SKIPPED with reason "upstream failed"
        break
    await asyncio.gather(*(run_task(t) for t in ready))    # capped at 8
```

For each task, the Scheduler routes by kind to the right worker, starts a `task.started` SSE event, runs the worker, and persists the result.

Outputs up to 8 KB live inline in the task row. Anything larger spills to `./artifacts/{session_id}/{task_id}.json` or `.html`, and the task stores only a preview plus the artifact path. This keeps downstream tasks cheap to read and the event bus payloads small.

Retry policy is in `app/engine/scheduler.py` and uses a deny list of fatal error patterns.

```
_FATAL_HINTS = (
    "ValueError", "not found", "missing",
    "TypeError", "SyntaxError", "NameError", "AttributeError",
    "ImportError", "ModuleNotFoundError", "IndentationError",
    "PolicyViolation", "violates sandbox policy",
)
```

Anything that matches a fatal hint is non retrievable. The rationale is that running the same buggy generated code three times will produce the same `TypeError` three times. Burning retries on deterministic failures just delays the inevitable replan. Truly transient failures (network glitches, MCP server hiccups, timeouts) still retry up to `task.max_retries` with exponential backoff (one second, then two, then four).

After the scheduler returns, the orchestrator inspects the final task statuses. If anything failed and there is still downstream work to do, it triggers one bounded replan. The Planner receives the prior failure including the saved checkpoint and produces a revised plan. The replan cap is set to one per session, because unbounded replan is a loop trap and one extra try is usually enough to recover from a code generation bug.

MCP orchestration strategy

The MCP layer is what the agent talks to. It exposes the four tools the assessment requires

(`get_document_insights`, `get_active_documents_metadata`, `translate_document_preserving_structure`, `aiagent`)
plus one bonus tool I added (`search_documents`, an FTS5 BM25 hybrid search for finding docs by keyword).

The agent never holds a long lived MCP session. Each call opens a streamable HTTP MCP session, runs one tool, and closes the session. The session ids in the MCP server logs are short lived by design. Connection overhead is negligible compared to tool latency.

There are two MCP call paths in the system.

The ToolWorker path is for single tool calls. It opens a session, calls the tool, parses the response, emits `task.mcp_call` and `task.mcp_result` events, and closes the session. That is it.

The CodeWorker path is more interesting. It generates a small Python script that imports a synthetic `mcp` module. That module is injected into the sandbox subprocess as `mcp_shim`, which routes any `mcp.<tool>(...)` call to a real MCP session over the network. From the generated code's perspective it looks like local function calls. From the parent process's perspective, every call emits an `##MCP_CALL##` marker on stdout that becomes a `task.mcp_call` SSE event. This is what lets bulk operations stream their MCP calls to the UI in real time.

The Planner prompt has explicit guidance on when to push filters into the MCP server (server side filtering with `category=`, `language=`, `status=` parameters) versus when to refine in Python. The rule is to push the primary filter dimension to the server (which translates to a SQL `WHERE` clause and dramatically reduces data transfer at scale) and refine secondary predicates in Python after the candidate set is already small.

State management

State lives in SQLite with WAL mode enabled. The schema has six tables.

```
sessions(id, container_id, status, created_at, user_msg,
         final_answer, webhook_url)
plans(id, session_id, goal, json_blob, created_at)
tasks(id, plan_id, kind, title, depends_on_json, spec_json,
      status, attempts, output_blob, artifact_ref, error,
      started_at, ended_at, checkpoint_json)
events(id PK autoinc, session_id, ts, type, payload_json)
questions(id, session_id, text, options_json, answer,
          asked_at, answered_at)
checkpoints(id, session_id, task_id, output_ref, created_at)
```

WAL mode means reads do not block writes. The event log is append only and never updated, which is what enables the SSE replay from cursor pattern. Late subscribers can pass `?cursor=N` and pick up exactly where a previous client left off.

Two columns are added via `ALTER TABLE` in `Store.connect()` with duplicate column errors swallowed, so older demo databases auto migrate on first run without any manual migration step.

The two columns are `sessions.webhook_url` (optional URL the orchestrator POSTs the final answer to) and `tasks.checkpoint_json` (the durability anchor for resume on startup, see next section).

Long running task handling

The naive design would set a hard wall clock timeout on every task. For a one million document translation that runs at one minute per document, that wall clock budget would need to be sixteen hours. Setting a sixteen hour timeout on every task has its own problem, since a genuinely hung script can sit doing nothing for sixteen hours before anyone notices.

The approach taken here is a heartbeat watchdog. For bulk task kinds (CODE_TRANSFORM and BULK_TOOL_CALL), the sandbox subprocess is killed only when no marker has appeared on stdout for

`sandbox_heartbeat_timeout_seconds` (default 300 seconds). The wall clock timeout still exists, but it is a 24 hour backstop, not the real liveness check. A real 17 hour translation runs to completion as long as something is happening every five minutes. A hung script dies in five minutes.

Non bulk tasks (TOOL_CALL, RAG_QUERY) keep the simple deadline because they are one MCP call or one LLM call. There is nothing to heartbeat against.

The heartbeat is implemented as a separate `_watchdog` coroutine in `code_worker.py` that polls every five seconds. Every `##PROGRESS##`, `##MCP_CALL##`, `##CHECKPOINT##`, or `##LOG##` line bumps the activity timestamp. The watchdog raises a `_SandboxLivenessError` if either the heartbeat threshold or the hard ceiling is exceeded.

Bulk execution strategy

This section covers how the system handles bulk operations across the corpus, which is the scalability axis the assessment evaluates. The naive failure mode is to let the planner LLM enumerate document IDs into its own context, or to let the orchestrator loop over docs one tool call at a time. The architecture prevents both, structurally.

The mechanism is the 20 doc rule combined with CODE_TRANSFORM. Anything that iterates over more than twenty documents becomes a CODE_TRANSFORM task. The CodeWorker generates a small Python script (typically around 150 lines), validates it via AST policy plus a compile pre check, and runs it in a sandbox subprocess. The script calls MCP tools directly through the `mcp` shim and streams progress.

A representative generated script for "translate all financial documents to German" looks like this.

```

import asyncio
import mcp

async def main():
    containers = __available_containers__ or [__container_id__]
    SERVER_FILTER = {"category": "financial"}
    LANG_TARGET = "deu"

    _emit_plan(
        [f"fetch metadata x{len(containers)} in parallel",
         f"bulk translate x{len(containers)} in parallel"],
        {"metadata_fetch": len(containers),
         "translate": len(containers),
         "mode": "asyncio.gather"},
    )

    metas = await asyncio.gather(*[
        mcp.get_active_documents_metadata(c, **SERVER_FILTER)
        for c in containers
    ])

    by_container = {}
    for meta in metas:
        ids = [d["documentId"] for d in meta["documents"]]
        if ids:
            by_container[meta["container_id"]] = ids

    async def translate_one(cid, ids):
        return await mcp.translate_document_preserving_structure(
            document_id=ids,
            destinationLanguageThreeLetterCode=LANG_TARGET,
            container_id=cid,
        )

    results = await asyncio.gather(*[
        translate_one(c, ids) for c, ids in by_container.items()
    ])

    successful = sum(r.get("successful", 0) for r in results)
    failed = sum(r.get("failed", 0) for r in results)
    failed_docs = [d for r in results for d in r.get("failed_documents", [])]

    _emit_result({
        "successful": successful,
        "failed": failed,
        "failed_documents": failed_docs[:50],
    })

```



```
asyncio.run(main())
```

What is happening here that scales.

The agent makes four LLM calls total regardless of corpus size, since the script generation is one LLM call and the rest is data movement between MCP and Python. For one million documents across four containers, the agent layer makes roughly 104 MCP HTTP calls (around 100 paginated metadata fetches plus one bulk translate per container with the full ID list). Inside the MCP server, the translate tool then runs all million translations internally with `Semaphore(200)`. The heavy lifting is at the tool, not at the agent.

The document IDs flow as strings through the Python script. Document content never crosses the agent boundary because the translate tool takes IDs and returns URLs to translated outputs in storage. At one million IDs (around 30 characters each), that is around 30 MB of ID strings in the sandbox process at peak, which is well within bounds.

Failure recovery

There are three kinds of failure the system has to handle, and each has its own recovery path.

The first is per document failure inside a bulk operation. The MCP translate tool has a hardcoded simulated three percent failure rate, which matches what a real translation pipeline looks like (some docs fail because they are corrupt, in an unexpected format, or have a processing timeout). The bulk MCP call returns success with `{successful: 970, failed: 30, failed_documents: [...]}`. No exception is raised. The script aggregates the per container responses, the task output captures the failure list, and the Synthesizer renders it in the final answer as a clean markdown table with the first twenty failed IDs listed in a code block. This is the "partial success reporting" the assessment calls out.

The second is whole task failure. The MCP server crashes, the network drops, the sandbox crashes, the generated code has a bug. The Scheduler catches the exception and decides whether to retry based on the `_FATAL_HINTS` deny list. Deterministic bugs (TypeError, SyntaxError, and the rest) go straight to the replan path. Transient failures retry up to two times with exponential backoff. After max retries, the task is marked FAILED and downstream tasks transitively skip.

The third is process death. The orchestrator runs in process and a uvicorn restart (deploy, crash, OOM) would normally lose all in flight work. To survive that, every `##CHECKPOINT##` marker is persisted to `tasks.checkpoint_json` immediately (not just at task end). On FastAPI startup, the lifespan calls `orchestrator.resume_interrupted_sessions()` which finds sessions in EXECUTING or PLANNING status, promotes their RUNNING tasks to a new INTERRUPTED status, and re executes them from their last checkpoint. The generated Python reads `__resume_from__` at the top of `main()` and skips already processed pages.

A 1M doc job that was 200,000 docs in when the process died resumes at doc 200,000 plus chunk_size on the next start. The user sees a `session.resumed` event and the original SSE, polling, and webhook channels continue as if nothing happened.

Retries

Retries are a careful design choice, not a default.

Transient failures retry with exponential backoff because the next attempt has a real chance of succeeding. The backoff schedule is one second, two seconds, four seconds, with `task.max_retries` set to two (so three attempts total).

Deterministic failures are never retried. The deny list in `_FATAL_HINTS` covers domain errors (`ValueError`, "not found", "missing") and Python level bugs in generated code (`TypeError`, `SyntaxError`, `NameError`, `AttributeError`, `ImportError`, `IndentationError`, `PolicyViolation`). All of these are guaranteed to fail again if retried with the same input. The recovery path is to replan with the failure context, not to retry.

Replanning is itself capped at one per session via `MAX_REPLANS = 1`. The replan path can be a loop trap if unbounded. One replan is usually enough to fix a code generation bug because the Planner sees the error text and chooses a different approach. Beyond one replan, the system surfaces the failure to the user with the full task error rather than spinning forever.

The webhook delivery has its own retry policy. If `webhook_url` is set on session creation, the orchestrator POSTs the final answer payload to it on `session.completed` or `session.error`. Webhook failures retry up to two times with a 10 second timeout, then give up silently. The SSE and polling channels are the primary delivery surfaces, the webhook is fire and forget for clients that have moved on.

Observability

Every interesting thing in the system emits a typed SSE event with a JSON payload. There are around 25 event types covering session lifecycle, plan creation and repair, task lifecycle, MCP call dispatch, code generation, sandbox stdout and stderr, progress markers, checkpoints, and the final answer. The complete list lives in `app/api/sse.py` alongside the encoder.

Three properties of the event stream are worth noting.

They are persisted to SQLite (append only) and assigned a monotonic id per session. The id becomes the SSE `id:` field on the wire. A late subscriber can pass `?cursor=N` and get every event after `N` in order, which means SSE clients can survive reconnects without losing events.

They are emitted from a per session asyncio queue plus a persistent log. Live subscribers get events through the queue (sub millisecond fan out). Late subscribers replay from the log first, then tail the queue. The same event is delivered exactly once to each subscriber.

They are typed and small. Each event has `{ts, payload}` in the `data:` field and the event `type` in the `event:` field. Payloads carry only the data needed for the UI, never full task outputs. Large outputs are spilled to `artifacts/` and referenced by path.

For server side observability, Loguru writes structured logs to stdout including Anthropic prompt cache hit rates (the `cache_read_input_tokens` and `cache_creation_input_tokens` fields are logged per Planner call). That makes it easy to see in production whether the cache is working as expected.

For client side observability, the `/ui` viewer subscribes to every event type and renders each appropriately. The full event log is in a collapsible panel at the bottom. Power users can `curl -N` the SSE endpoint and pipe it to `jq` for shell debugging.

Part 2, Working Code

This part walks through what is actually running and what it looks like from the outside.

Planning, in code

The Planner is implemented in `app/engine/planner.py`. The hot path is short.

```

async with self.client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={"type": "adaptive"},
    output_config={"effort": "high"},
    cache_control={"type": "ephemeral"},
    system=PLANNER_SYSTEM,
    tools=[EMIT_PLAN_TOOL],
    tool_choice={"type": "auto"},
    messages=[{"role": "user", "content": user_block}],
) as stream:
    async for event in stream:
        if event.type == "content_block_delta":
            delta = event.delta
            if delta.type == "thinking_delta":
                await self.bus.emit(session_id, "planner.thinking",
                                   {"delta": delta.thinking})

    final_message = await stream.get_final_message()

# Extract the emit_plan tool call from the final message
plan_payload = None
for block in final_message.content:
    if block.type == "tool_use" and block.name == "emit_plan":
        plan_payload = block.input
        break

plan = Plan(
    session_id=session_id,
    goal=plan_payload["goal"],
    tasks=[coerce_task(t) for t in plan_payload["tasks"]],
)

```

The streaming loop forwards every `thinking_delta` to the SSE bus, which is how the Planner's reasoning surfaces in the UI as it is produced. The final tool call carries the structured plan. Everything else is plumbing.

Agent routing, in code

Agent routing is one stateless function in `app/engine/router.py`.

```

class Router:
    def route(self, task: Task) -> Worker:
        kind = task.kind
        if kind in (TaskKind.TOOL_CALL, TaskKind.RAG_QUERY):
            return ToolWorker()
        if kind in (TaskKind.CODE_TRANSFORM, TaskKind.BULK_TOOL_CALL):
            return CodeWorker()
        if kind in (TaskKind.SUBAGENT, TaskKind.SYNTHESIZE):
            return SubAgentWorker()
        raise ValueError(f"unknown task kind: {kind}")

```

That is the whole router. There is no agent registry, no plugin system, no dynamic dispatch. The six task kinds map deterministically to three worker classes. Adding a new task kind means adding a new branch and maybe a new worker.

The reason the router is this small is that the Planner has already done the hard part. It picked the task kind, so the router just has to honour that pick.

MCP tool execution, in code

The ToolWorker is the simplest worker. One call.

```

class ToolWorker(Worker):
    name = "tool"

    async def execute(self, ctx, task):
        spec = task.spec
        if task.kind == TaskKind.RAG_QUERY:
            return await mcp_call_tool("aiagent", {
                "prompt": spec["prompt"],
                "container_id": spec.get("container_id") or ctx.container_id,
            })
        # TOOL_CALL
        tool = spec["tool"]
        args = spec.get("args") or {}
        return await mcp_call_tool(tool, args)

```

`mcp_call_tool` lives in `app/mcp_client/client.py`. It opens a fresh streamable HTTP MCP session, calls the tool, parses the response (handling both structured content and text fallback), emits SSE events for the call and the result, and closes the session.

The CodeWorker is more involved. It calls `_generate_script` to get the script string, runs `check_script` for AST policy validation, runs `compile(script, "<sandbox>", "exec")` for the syntax pre check, and then launches a subprocess with the script and runs the heartbeat watchdog. The sandbox executes the script with `mcp rebound`

to the shim that routes calls back to the real MCP server over HTTP. Every MCP call inside the sandbox emits `##MCP_CALL##` and `##MCP_RESULT##` markers that become SSE events.

Multi step workflows

A multi step workflow is just a plan with more than one task. The Scheduler walks the DAG, dispatches each ready task, waits for the wave to complete, and moves on. Dependencies are pure data references, not orchestration glue.

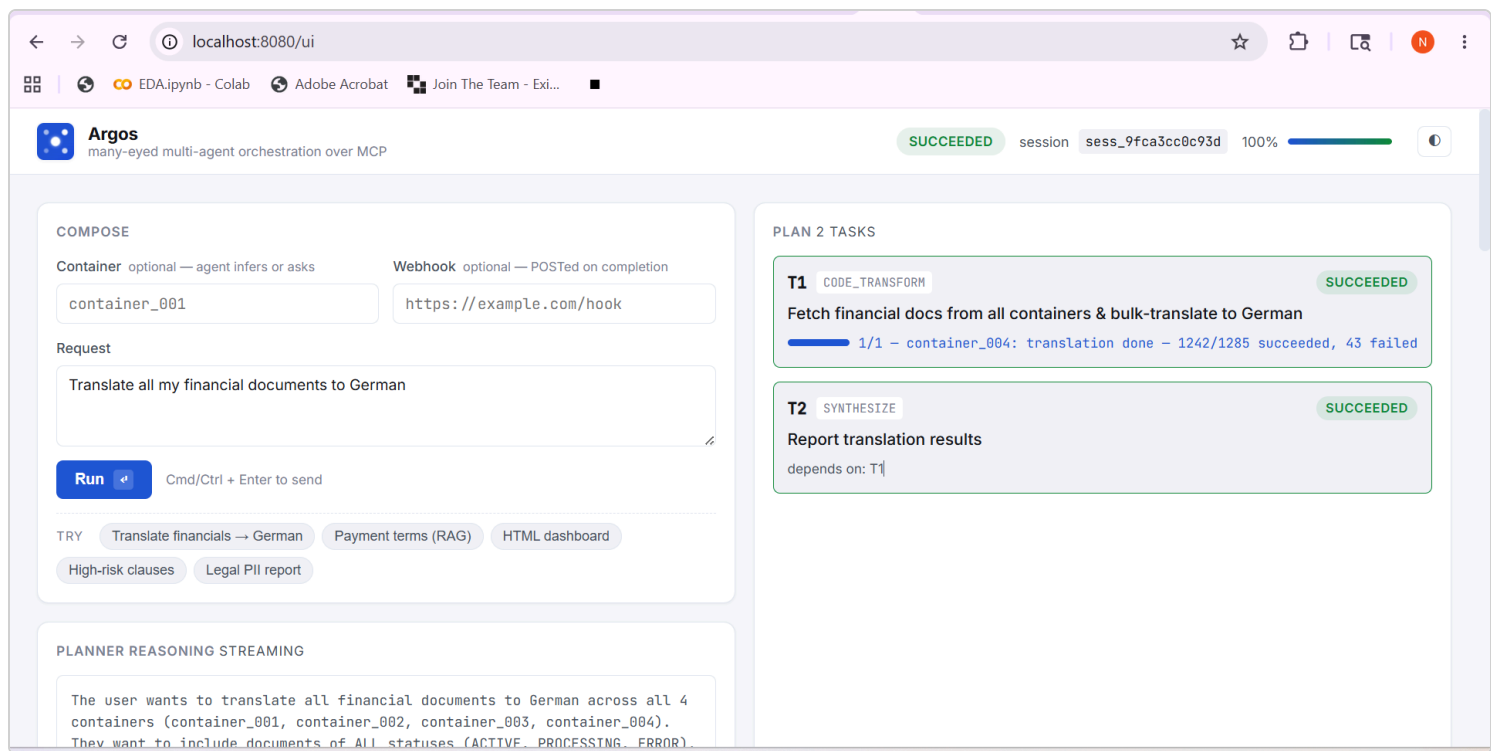
Here is what happens for "translate all financial documents to German" end to end, with the LLM and MCP call counts.

1. POST /v1/sessions (create session, no LLM)
2. POST /v1/sessions/{id}/messages
 - 2a. Interrogator (1 LLM call: proceed)
 - 0 MCP calls, since the only container check is in memory
 - 2b. Planner (1 LLM call: emit_plan)
 - 1 MCP call: list_tools to build the dynamic prompt
 - 2c. validate_and_repair (0 LLM calls, in process)
 - 2d. Scheduler runs T1 CODE_TRANSFORM
 - 2d.1. CodeWorker generate_script (1 LLM call)
 - 2d.2. compile pre check (in process)
 - 2d.3. Sandbox subprocess runs the script
 - 4 MCP calls: get_active_documents_metadata per container
 - 4 MCP calls: translate_document_preserving_structure per container (each carries the full bulk ID list as one argument)
 - 2d.4. ##RESULT## marker parsed, task complete
 - 2e. Scheduler runs T2 SYNTHESIZE
 - 2e.1. SubAgentWorker (1 LLM call: compose markdown answer)
 - 2f. Orchestrator emits session.completed (0 LLM calls)
 - Optional webhook POST (0 LLM calls)
3. Client streams events via SSE the entire time

That is 4 LLM calls and 9 MCP calls for the entire workflow. The million translations live inside the MCP server's translate tool behind `Semaphore(200)`. The agent layer never sees them individually.

Sample UI

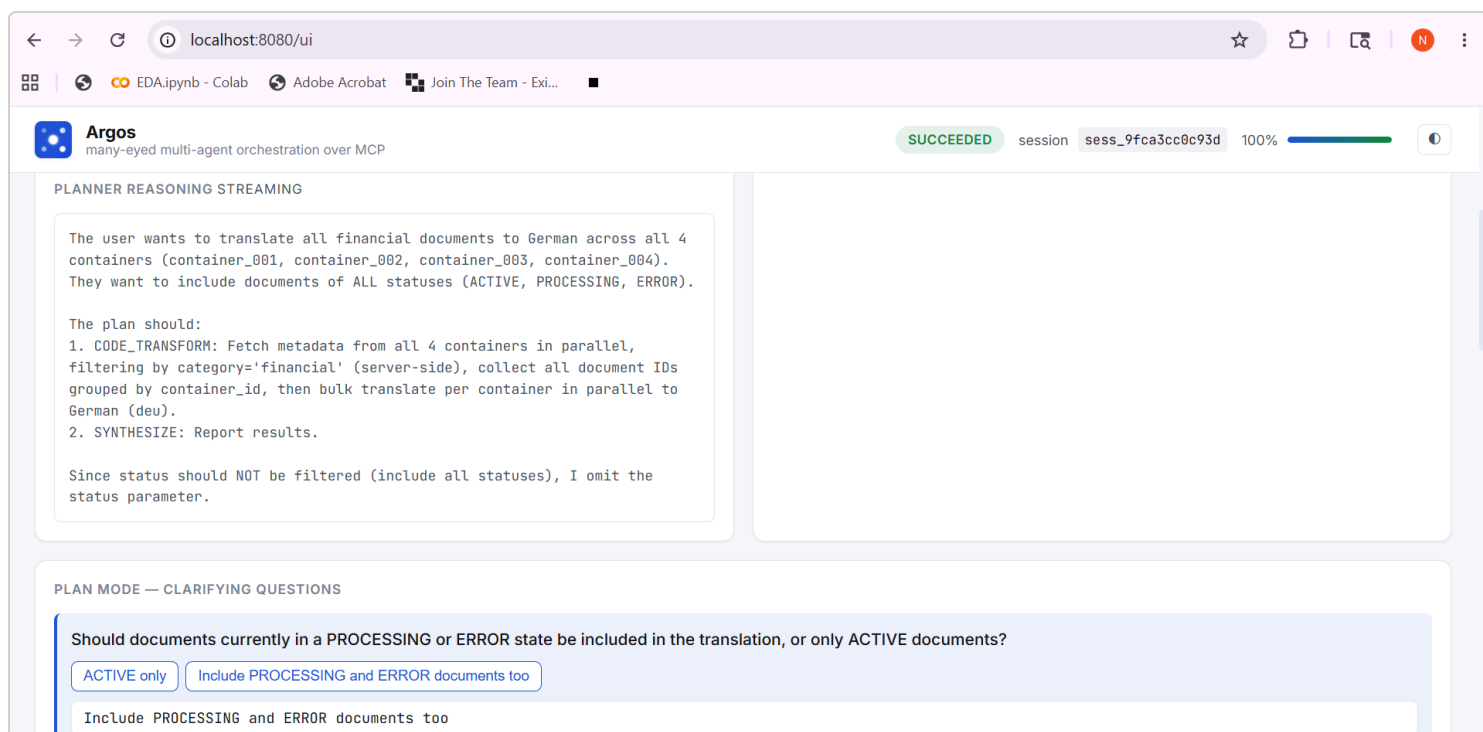
The UI lives at `http://localhost:8080/ui`. It is a single page HTML viewer with no build step, no framework, no bundler. One HTML file, one CSS file, and one JS file consume the SSE stream and render the session as it unfolds.



Argos main UI in SUCCEEDED state. The Compose panel on the left holds the container input, the optional webhook URL, the request text area, the Run button, and the suggested query chips for the six PDF example queries. The Plan panel on the right shows the resolved two task plan. T1 is a TOOL_CALL labelled "Fetch financial docs from all containers and bulk translate to German" marked SUCCEEDED, with the live counter showing 1242 of 1285 succeeded and 43 failed. T2 is a SYNTHESIZE task labelled "Report translation results" that depends on T1 and is also SUCCEEDED. The planner reasoning panel at the bottom contains the streamed thinking the model produced during planning.

The Plan panel renders the DAG as a vertical list of task cards. Each card carries the task id, the kind badge, the title, an optional dependency line, a status pill that updates live, and a progress bar for tasks that emit `##PROGRESS##` markers. The progress on T1 above came from the CodeWorker's generated script translating the financial documents one bulk MCP call per container in parallel.

Plan Mode is the interrogation flow that catches ambiguous requests before they reach the Planner. The screenshot below is from the query "translate all my financial documents to German" which is ambiguous about whether to include documents currently in PROCESSING or ERROR status alongside ACTIVE ones.



Plan Mode clarifying question. Above the question, the planner reasoning streaming panel is visible with text explaining the user wants to translate all financial documents to German across all four containers, that the plan should fetch metadata, filter by category, group by container, and bulk translate per container. Below the streaming panel, a blue framed clarifying question reads "Should documents currently in a PROCESSING or ERROR state be included in the translation, or only ACTIVE documents?". Two option buttons are visible, ACTIVE only and Include PROCESSING and ERROR documents too. Below the buttons is a free text input. The user has already typed "Include PROCESSING and ERROR documents too" in the input, and a green check below confirms the answer was submitted.

When the user clicks an option or types a custom answer and presses Enter, the answer is POSTed to `/v1/sessions/{id}/answer` and the session resumes. The Planner re enters with the answer included in its context block. The session pill transitions from AWAITING back to PLANNING and then EXECUTING.

The final answer at the bottom of the UI is plain markdown rendered inline by a small client side renderer in `app/ui/app.js` . Tables, code blocks, inline code, links, and lists all render without external dependencies.

localhost:8080/ui

Argos
many-eyed multi-agent orchestration over MCP

SUCCEEDED session sess_9fca3cc0c93d 100%

FINAL ANSWER

Translation of 5,140 financial documents to German across four containers is complete.

Key Results

METRIC	COUNT
Total documents	5,140
Successful	4,980
Failed	160

BY CONTAINER

CONTAINER	FOUND	SUCCESSFUL	FAILED
container_001	1,285	1,245	40
container_002	1,285	1,246	39
container_003	1,285	1,247	38

Final Answer section, scrolled into view. The session pill in the header reads SUCCEEDED in green. The answer begins with the sentence "Translation of 5,140 financial documents to German across four containers is complete." Below it, a Key Results table has three rows. Total documents 5,140. Successful 4,980. Failed 160. Below that, a By Container section shows a second table breaking the totals down per container. container_001 found 1,285 with 1,245 successful and 40 failed. container_002 found 1,285 with 1,246 successful and 39 failed. container_003 found 1,285 with 1,247 successful and 38 failed.

The Synthesizer prompt holds the model to a tight markdown style. Clean tables with right aligned numbers, inline code for identifiers, no emoji, no backticks around URLs. Bare relative artifact paths get rewritten to absolute URLs by a post processor so the links resolve to the `/artifacts/` static route immediately.

The Event Log panel at the bottom of the UI shows every SSE event as it arrives, including the clickable artifact link when a task produces an HTML report.

localhost:8080/ui

Argos
many-eyed multi-agent orchestration over MCP

SUCCEEDED session sess_9fca3cc0c93d 100%

container_001_doc_002006

The full list of 160 failed IDs is available in the report below.
Report: http://localhost:8080/artifacts/sess_9fca3cc0c93d/T2.html

EVENT LOG 52 events

05:31:46	task.code_progress	{"current":1,"total":1,"msg":"container_003: translation done - 1247/1285 succeeded, 38 failed","task_id":"T1"}
05:31:46	task.mcp_result	{"id":6,"tool":"translate_document_preserving_structure","ok":true,"summary":{"mode":"bulk","total":1285,"successful":1246,"fa...
05:31:46	task.code_progress	{"current":1,"total":1,"msg":"container_002: translation done - 1246/1285 succeeded, 39 failed","task_id":"T1"}
05:31:46	task.mcp_result	{"id":8,"tool":"translate_document_preserving_structure","ok":true,"summary":{"mode":"bulk","total":1285,"successful":1242,"fa...
05:31:46	task.code_progress	{"current":1,"total":1,"msg":"container_004: translation done - 1242/1285 succeeded, 43 failed","task_id":"T1"}
05:31:49	task.completed	{"task_id":"T1","output_preview":{"total_found":5140,"successful":4980,"failed":160,"failed_documents":["container_001_doc_001...
05:31:49	task.started	{"task_id":"T2","kind":"SYNTHESIZE","worker":"subagent","title":"Report translation results"}
05:31:49	subagent.spawned	{"task_id":"T2","role":"specialist"}
05:32:24	task.completed	{"task_id":"T2","output_preview":{"kind":"html_artifact","artifact_ref":"artifacts/sess_9fca3cc0c93d/T2.html","size_bytes":987...
05:32:31	session.completed	{"final_answer":"Translation of 5,140 financial documents to German across four containers is complete.\n\n## Key Results\n\n ...

Event Log panel. The text just above it says "The full list of 160 failed IDs is available in the report below. Report: http://localhost:8080/artifacts/sess_9fca3cc0c93d/T2.html" with the URL rendered as a clickable link. The event log header shows EVENT LOG with a count of 52 events on the right. Visible rows include task.code_progress with a payload showing translation totals, task.mcp_result with the translate tool's response, task.completed with the SYNTHESIZE task id, task.started for the synthesize task, subagent.spawned with the kind HTML, task.completed referencing the artifact ref, and session.completed at the very end carrying the final answer.

Every event is typed, persisted, and replayable from a cursor on reconnect. Power users can `curl -N` the SSE endpoint and pipe it to `jq` for shell debugging. UI users see the same data rendered as task cards, progress bars, and the event log panel.

Part 3, Design Decisions

Why this architecture

The assessment explicitly evaluates whether the system thinking is sound. Using a higher level framework like LangGraph or LlamaIndex hides the planner, router, and worker decisions inside someone else's abstraction. Writing the DAG executor and worker dispatch ourselves shows the architecture more clearly. We do use the official MCP Python SDK for the wire format, since there is no reason to reinvent that.

The three load bearing design choices.

First, the planner sees only document IDs, never document content, and never more than O(1) IDs at once. Everything iterative becomes a CODE_TRANSFORM task whose generated Python runs in a sandbox subprocess and calls MCP tools directly. This is the scalability invariant. Without it, the agent's context grows linearly with the

corpus and the system falls over at around fifty thousand documents. With it, the agent's context is constant whether the corpus is nine thousand or one million documents.

Second, the LLM call count is fixed at four per session (Interrogator, Planner, Code generation, Synthesizer) regardless of corpus size. The Anthropic API bill scales with conversation complexity, not with corpus size. A two task plan that touches one million documents costs roughly the same as a two task plan that touches ten documents.

Third, every operation that can take more than a few seconds is durable end to end. The event log is append only and replayable from a cursor. Bulk operations checkpoint after every chunk. Sessions left in EXECUTING at process death are detected on the next startup and resumed from their last checkpoint. The orchestrator process is fungible. State lives in SQLite.

These three choices are the architecture's structural commitments. The rest is implementation detail.

Tradeoffs considered

A few choices were not obvious and deserve mention.

I chose a hard wall clock timeout originally, then replaced it with a heartbeat watchdog. The hard timeout was easier to implement (one `asyncio.wait_for` call) but it forced an impossible choice between killing real long running jobs and letting hung scripts hang forever. The heartbeat watchdog handles both cases correctly at the cost of one more coroutine and some shared mutable state for the last activity timestamp.

I chose to retry transient failures and not retry deterministic failures. Retrying a `TypeError` three times produces the same `TypeError` three times, which wastes ten to twenty seconds of latency and generates noisy logs. The deny list in `_FATAL_HINTS` is the implementation. The cost is that occasionally a non Python error message contains the word "TypeError" coincidentally and gets incorrectly classified. In practice this has not happened.

I chose to give the Planner a single tool (`emit_plan`) with `tool_choice=auto` rather than forcing the tool call. Forced tool use is incompatible with adaptive thinking, and adaptive thinking empirically produces better plans than a fixed thinking budget. The tradeoff is that occasionally the model could choose to emit text instead of calling the tool. The system prompt is explicit that the tool must be called, and in many runs this has not happened. If it did, the orchestrator would surface a clear error rather than silently misbehaving.

I chose subprocess isolation over in process execution for generated code. In process is faster (no fork overhead) but it shares the event loop and the global Python state with the parent, which means a bad generated script could pollute or hang the API process. Subprocess isolation costs around 200 ms per task launch and gives clean failure semantics (kill the process, everything dies, no cleanup needed).

I chose to validate the plan after generation rather than constraining the schema more tightly. Tighter constraints would make the `emit_plan` tool schema harder to use for the LLM. Looser constraints with post hoc validation lets the LLM stay productive while the `validate_and_repair` function catches the rare violations.

Why I avoided alternative approaches

I avoided LangGraph and LangChain. The assessment is about system design, and using a higher level framework hides exactly the decisions the assessment is grading. LangGraph in particular forces a graph based execution model that does not fit the topological wave executor I wanted. The Send API in LangGraph for parallel branches is awkward for DAGs with arbitrary fan in.

I avoided LlamaIndex and CrewAI for similar reasons. They are great frameworks for building applications quickly, but they wrap the LLM calls in their own abstractions and the reviewer would have to read through those abstractions to see how the actual orchestration works.

I avoided a SQL agent that generates arbitrary SQL queries against the document corpus. SQL agents work but have no structural protection against accidentally broad queries or slow full table scans. The MCP tool abstraction protects the schema and limits blast radius. The existing `search_documents` MCP tool (FTS5 BM25 plus structured filters) covers the keyword search cases this assessment exercises.

I avoided Docker in Docker for the sandbox. Docker in Docker doubles the build time and complicates the filesystem. AST allowlist plus filtered builtins covers the trust model assumed in a demo setting, where the LLM is the trusted but confused party and not an attacker. For production with untrusted code, E2B or Firecracker would be a closer match to the threat model.

I avoided E2B for the demo because it is a paid third party service that adds external latency for every code execution. The subprocess sandbox is local and fits the demo trust model described above.

I avoided unbounded replan after task failure. Replan can loop forever if the Planner keeps making the same mistake. One bounded replan covers the common case (Planner sees the error, picks a different approach, succeeds) without the loop risk.

I avoided custom MCP transport. The official MCP Python SDK speaks streamable HTTP, that is the protocol the assessment's `Sample_FastMCP.py` uses, and there is no value in reimplementing it.

Scaling considerations

This section accounts for what changes between 36,000 documents and one million in this architecture.

The agent layer changes nothing. Four LLM calls per session. $O(1)$ document IDs in the Planner's context. The same DAG shape, the same worker dispatch.

The MCP HTTP call count grows logarithmically with corpus size, not linearly. At 36K docs across 4 containers, around 10 MCP calls per "translate all financial documents" workflow. At 1M docs across 4 containers, around 104 MCP calls (100 paginated metadata fetches plus 4 bulk translate calls). The bulk translate carries the full ID list in one HTTP request, which is roughly 30 MB at one million IDs and fits comfortably in one HTTP body.

The MCP server's internal work scales linearly with corpus size by design, since that is where the actual document operations happen. At 0.02 seconds per simulated translation with `Semaphore(200)`, one million docs takes around 85 seconds wall clock time. To compress wall clock time on real workloads, scale the MCP server horizontally with multiple replicas behind nginx round robin. The tool is stateless reads plus write once outputs, so replicas work without coordination.

The state DB stays on SQLite up to around 5 to 10 million events. Beyond that, swap `app/state/store.py` for a Postgres implementation. The Store abstraction is the only seam that needs to change.

The event volume per session is bounded by chunks and markers, not by document count. A 1M doc translation emits roughly 500 to 1500 events per session, depending on chunking. That fits in any event log implementation.

Memory in the agent process stays in the low tens of MB regardless of corpus size, because document content never crosses the agent boundary. Outputs over 8 KB spill to `artifacts/` and only IDs and counts flow through the event bus.

Wall clock time at scale follows the MCP server's concurrency limit, not the agent's. The agent submits the work and gets out of the way.

Production concerns

This codebase is a demo, not a production deployment. The agent layer covers the assessment requirements end to end, but a real production rollout would still need standard operational hardening that lives outside the agent layer.

The five things I would prioritise for production are per tenant authentication scoped to container ids in front of `/v1/`, swapping the subprocess sandbox for a stronger isolation layer like E2B or Firecracker if generated code becomes untrusted, moving session and event state from SQLite to Postgres with Redis as a fast cache for in flight work, wiring cost telemetry from the Anthropic `usage` blocks into OpenTelemetry spans for per session attribution, and adding multi turn conversation history within a session so the agent can refine across messages instead of starting fresh each time.

The current architecture absorbs all of these without restructuring. The Store abstraction in `app/state/store.py` is the seam for the state backend swap. The Anthropic client wrapper is the seam for telemetry. Auth is a FastAPI middleware. Sandbox is one worker module. Each change is localised, not a rewrite.